

Universidad Tecnológica Nacional
Facultad Regional Concepción del Uruguay
Ingeniería en Sistemas de Información

Gestión Avanzada de Datos

Trabajo Práctico III
Optimización de consultas

Alumnos:

- Brun Juan
- Riedel Manuel Ángel
- Leandro Guiffrey

Docentes:

- Mg. Ing. Andrés J. Pascal
- Mg. Ing. Planas, Adrián Nicolás

a. Resolver las siguientes consultas utilizando Explain Analyze y describir los planes de
ejecución.

1. Devolver todos los comercios que se encuentran en la calle Mitre

```
Seq Scan on comercios c  (cost=0.00..83.31 rows=46 width=281) (actual time=0.046..0.795
rows=46.00 loops=1)
  Filter: ((calle)::text = 'Mitre'::text)
  Rows Removed by Filter: 1979
  Buffers: shared hit=58
Planning Time: 0.229 ms
Execution Time: 0.848 ms
```

Seq Scan on comercios c: postgre recorrió toda la tabla comercios fila por fila para buscar los registros que cumplen la condición.

2. Crear un índice hash por calle para los comercios y ejecutar la consulta anterior y comparar los resultados y el plan de ejecución

```
Bitmap Heap Scan on comercios c  (cost=4.36..62.26 rows=46 width=281) (actual
time=0.068..0.174 rows=46.00 loops=1)
  Recheck Cond: ((calle)::text = 'Mitre'::text)
  Heap Blocks: exact=33
  Buffers: shared hit=35
  ->  Bitmap Index Scan on idx_comercios_calle_hash  (cost=0.00..4.34 rows=46 width=0)
(actual time=0.030..0.031 rows=46.00 loops=1)
        Index Cond: ((calle)::text = 'Mitre'::text)
        Index Searches: 1
        Buffers: shared hit=2
Planning:
  Buffers: shared hit=15
Planning Time: 0.361 ms
Execution Time: 0.231 ms
```

Bitmap Heap Scan on comercios c: PostgreSQL encontró primero las filas mediante el índice y después fue a la tabla comercios para obtener los datos completos.

3. Devolver la cantidad de comercios agrupados por producto principal

```
HashAggregate  (cost=88.38..91.20 rows=283 width=21) (actual time=1.432..1.493
rows=283.00 loops=1)
  Group Key: productoprincipal
  Batches: 1  Memory Usage: 56kB
  Buffers: shared hit=58
  ->  Seq Scan on comercios c  (cost=0.00..78.25 rows=2025 width=13) (actual
time=0.022..0.337 rows=2025.00 loops=1)
        Buffers: shared hit=58
Planning Time: 0.136 ms
```

Execution Time: 1.573 ms

4. Crear un índice B-Tree por producto principal y ejecutar la consulta anterior y comparar los resultados y el plan de ejecución

GroupAggregate (cost=0.28..63.61 rows=283 width=21) (actual time=0.216..2.808 rows=283.00 loops=1)
  Group Key: productoprincipal
  Buffers: shared hit=1 read=4
  -> Index Only Scan using idx_comercio_ppincipal on comercios c (cost=0.28..50.65 rows=2025 width=13) (actual time=0.192..1.311 rows=2025.00 loops=1)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=1 read=4
Planning:
  Buffers: shared hit=18 read=1
Planning Time: 0.718 ms
Execution Time: 2.961 ms

Para esta tabla de 2025 registros, el plan sin índice presentó un menor tiempo de ejecución (1,573 ms frente a 2,961 ms), por lo que el uso del índice no resultó beneficioso en esta consulta específica.

5. Crear un índice Hash por producto principal y ejecutar la consulta anterior y comparar los resultados y el plan de ejecución

Bitmap Heap Scan on comercios c (cost=4.36..62.26 rows=46 width=281) (actual time=0.841..0.974 rows=46.00 loops=1)
  Recheck Cond: ((calle)::text = 'Mitre'::text)
  Heap Blocks: exact=33
  Buffers: shared hit=33 read=2
  -> Bitmap Index Scan on idx_comercios_calle_hash (cost=0.00..4.34 rows=46 width=0) (actual time=0.777..0.777 rows=46.00 loops=1)
        Index Cond: ((calle)::text = 'Mitre'::text)
        Index Searches: 1
        Buffers: shared read=2
Planning:
  Buffers: shared hit=244 read=10
Planning Time: 7.498 ms
Execution Time: 1.091 ms

Comparando ambas ejecuciones, el plan se mantiene igual: PostgreSQL utiliza el índice Hash mediante un Bitmap Index Scan y luego obtiene los datos con un Bitmap Heap Scan.

En la primera ejecución el tiempo fue de 0,231 ms, mientras que en la segunda fue de 1,091 ms. Aunque esta vez tardó más, la diferencia puede deberse a factores de caché y carga del sistema. En ambos casos se encontraron las mismas 46 filas y se utilizó el índice correctamente.

6. Devolver todos los comercios que se encuentran en el barrio 1

```
select * from comercios c  where c.barrio = 1;

Seq Scan on comercios c  (cost=0.00..83.31 rows=3 width=281) (actual time=0.711..0.970
rows=3.00 loops=1)
  Filter: (barrio = 1)
  Rows Removed by Filter: 2022
  Buffers: shared hit=58
Planning Time: 0.255 ms
Execution Time: 1.014 ms
```

7. Crear un índice hash para mejorar la consulta anterior y comparar los resultados y el plan de ejecución

```
Bitmap Heap Scan on comercios c  (cost=4.02..14.01 rows=3 width=281) (actual
time=0.065..0.075 rows=3.00 loops=1)
  Recheck Cond: (barrio = 1)
  Heap Blocks: exact=3
  Buffers: shared hit=5
  ->  Bitmap Index Scan on idx_comercio_barrio  (cost=0.00..4.02 rows=3 width=0) (actual
time=0.032..0.032 rows=3.00 loops=1)
        Index Cond: (barrio = 1)
        Index Searches: 1
        Buffers: shared hit=2
Planning:
  Buffers: shared hit=15
Planning Time: 0.498 ms
Execution Time: 0.137 ms
```

8. Devolver todos los comercios que se encuentran en el barrio Centro, en calle galarza,
con clanae correpondiente al rubro Kiosko

9. Devolver los pares de comercios del mismo rubro (clanae) que están cerca uno del otro (misma calle y número +-50)

10. Devolver los comercios que se encuentran en la calle 8 de junio con clanae 47119

11. Crear un btree para comercios, con calle y clanae. Volver a ejecutar la consulta anterior y verificar si cambió el plan

12. Escribir 3 consultas que contengan joins, subconsultas y/o agregaciones, verificar el
plan de ejecución. Tratar de mejorar su eficiencia planteando la consulta de distinta manera y/o creando índices.

b. Dadas las siguientes tablas:

```
create table articulos as
select id::int8, random_string(100) as descripcion, round(random() * 30)::int8 as rubro
from generate_series(1, 500000) as id;
create table precios as
select id::int8,
round(random() * 99000)::int8+1 as id_articulo,
timestamp '2022-01-01 00:00:00' + (random() * (interval '1000 days')) as fecha,
100.00+round(random() * 5000)::numeric(11,2) as precio
from generate_series(1, 8000000) as id;
```

- la funcion random_string está dada por:

```
CREATE OR REPLACE FUNCTION public.random_string(
length integer) RETURNS text LANGUAGE 'plpgsql'
AS $BODY$
declare
chars text[] :=
'{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z,a,b,c,d,e,f,g,h,i,j,k
,l
,m,n,o,p,q,r,s,t,u,v,w,x,y,z}';
result text := '';
i integer := 0;
begin
if length < 0 then
raise exception 'Given length cannot be less than 0';
end if;
for i in 1..length loop
result := result || chars[1+random()*(array_length(chars, 1)-1)];
end loop;
return result;
end;
$BODY$;
```

resuelva las consultas que se enuncian a continuación, explique su plan de ejecución e intente hacerlas más eficientes:

1. Artículos correspondientes al rubro 5
2. Precio actual del articulo 10
3. Precio actual y descripción del articulo 10
4. Artículos cuyos precios están entre 1000 y 1500
5. Precio promedio por articulo
6. Precio promedio por rubro
7. Máximo precio del rubro 15, posterior marzo de 2023
8. Maximo precio promedio por rubro
9. Máximo precio promedio por artículo

c. Crear una tabla con una gran cantidad de registros (más de 500,000 filas) que incluya al menos cuatro columnas con valores repetidos (por ejemplo, una columna de nombres, apellido, edad y ciudad)

1. Ejecutar “EXPLAIN ANALYZE SELECT * FROM personas WHERE ciudad = 'Colón';” y analizar y describir el plan de ejecución.
2. Crear un índice btree sobre la columna ciudad y volver a ejecutar y describir los resultados.
3. Eliminar el índice btree y crear un índice hash sobre la columna ciudad y volver a ejecutar y describir los resultados.
4. Realizar una consulta que busque las personas que estén dentro de un rango de edad, ejecutar EXPLAIN ANALYZE y describir los resultados
5. Crear un índice btree sobre la edad y volver a ejecutar y analizar.
6. Crear una consulta con funciones agregadas (count, sum, avg) y analizar el plan de ejecución con y sin índices sobre la columna de cálculo.
d. Dadas las siguientes relaciones

```
CREATE TABLE clientes (
id SERIAL PRIMARY KEY,
nombre TEXT,
email TEXT UNIQUE
);
CREATE TABLE pedidos (
id SERIAL PRIMARY KEY,
cliente_id INT REFERENCES clientes(id),
fecha DATE,
total NUMERIC
);
```

1. Insertar 500k clientes y 5M pedidos distribuidos aleatoriamente
2. Ejecutar una consulta para obtener los 10 clientes con mayor facturación en 2024

```
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;
```

3. Explicar el plan de ejecución
4. Probar índices con distintas claves en Pedidos:

```
INDEX(cliente_id)
INDEX(cliente_id, fecha)
INDEX(fecha, cliente_id)
```

5. Analizar cuál da mejor rendimiento y justificar.